数据类型简介和检测

1

JavaScript数据类型

- JavaScript的数据类型：5基多引

	类型名	值举例
基本类型值	数字	5
	字符串	'爱前端'
	布尔	true
	undefined	undefined
	null	null
引用类型值	数组	[33, 44, 11]
	对象	{a: 1, b: 2}
	函数	function(){}
	正则表达式	/\d{3}/
	……	

2

使用typeof检测数据类型

- 使用typeof运算符可以检测值或者变量的类型

```
console.log(typeof 5);          // number
console.log(typeof '爱前端');  // string

var a = 12;
var b = '爱前端';
console.log(typeof a);        // number
console.log(typeof b);        // string
```

3

typeof检测结果

	类型	typeof检测结果	值举例
基本类型值	数字	number	5
	字符串	string	'爱前端'
	布尔	boolean	true
	undefined	undefined	undefined
	null	object	null
引用类型值	数组	object	[33, 44, 11]
	对象	object	{a: 1, b: 2}
	函数	function	function(){}
	正则表达式	object	/\d{3}/
	……		

4

number（数字）类型

5

所有数字都是number类型

- 所有数字不分大小、不分整浮、不分正负，都是数字类型

```
console.log(typeof 5564234234234);  // number
console.log(typeof 2);              // number

console.log(typeof 234);            // number
console.log(typeof 2.34);           // number

console.log(typeof 5);              // number
console.log(typeof -5);             // number
```

6

小数的0可以省略

- 介于0和1之间的小数，可以不书写0

```
console.log(.5);          // 0.5
console.log(typeof .5); // number
```

7

科学计数法

- 较大数、较小数、绝对值较小数，都可以写成科学计数法

```
console.log(3e8);       // 300000000
console.log(-3e8);      // -300000000
console.log(3e-4);      // 0.0003
console.log(-3e-4);     // -0.0003
```

8

number类型的更多知识今后讲解

- number中有两个特殊值：NaN和Infinity
- 数字运算符
- JavaScript支持二进制、八进制、十六进制表示法，在学习俄罗斯方块的时候我们介绍它们

9

NaN和Infinity

10

特殊数字值NaN

- NaN是英语“not a number”的意思，即“不是一个数”，但它是一个数字类型的值

```
console.log(typeof NaN);     //number
```

11

什么时候会得到NaN

- 零除以零结果是NaN，非零除以零结果不是NaN而是Infinity
- 数学计算中，结果难以得到数字值的计算式，结果均为NaN
- NaN参与的数学运算，结果通常为NaN

```
console.log(0 / 0);              // NaN
console.log(3 * '西瓜');         // NaN
console.log('我' - '你');        // NaN
console.log(NaN * 3);            // NaN
```

注意'我'+'你'结果不是NaN哦，而是'我你'

12

特殊数字值Infinity

- Infinity是英语“无穷大”的意思，

13

特殊数字值Infinity

- Infinity是一个数字类型值，表示无穷大

```
console.log(typeof Infinity);   // number
```

14

什么时候会得到Infinity

- 非零数字除以0的结果是Infinity
- 当计算结果非常大或非常小时，系统将得到Infinity

```
console.log(4 / 0);          // Infinity
console.log(-4 / 0);         // -Infinity
console.log(9e99999);        // Infinity
console.log(-9e99999);       // -Infinity
```

15

NaN和Infinity后续内容介绍

- NaN和Infinity暂时介绍到这里，后续在“运算符”课程中还有一些关于它的内容
- 比如NaN不自等，再比如Infinity - Infinity等于多少……

16

string（字符串）类型

17

引号

- 字符串要用引号包裹，单引号或者双引号均可，越来越多的公司要求使用单引号

```
console.log(typeof '爱前端');   // string
console.log(typeof "爱前端");   // string
```

18

分清数字和字符串

- 数字11和字符串'11'在语义上是不同的，前者表达一个数量，后者是一个文本

```
console.log(typeof 11);      // number
console.log(typeof '11');    // string
```

19

字符串的拼接

- 加号可以用来拼接多个字符串

```
console.log('我是' + '一个' + '考拉');
```

20

字符串和变量的拼接

- 要将一个变量的值“插入”到字符串中，要“斩断链接”

```
var year = 2022;
var str = '北京冬季奥运会在' + year + '年召开';
```

```
var year = 2022;
var str = '北京冬季奥运会在year年召开';
```

```
var year = 2022;
var str = `北京冬季奥运会在${year}年召开`;
```

21

空字符串

- 一些时候需要使用空字符串，直接书写闭合的引号对即可

```
var str = '';
```

22

字符串的length属性

- 字符串的length属性表示字符串的长度，任何字符都占1位

```
console.log('我喜欢JS'.length);                    // 5
console.log('我喜欢JS，我也喜欢HTML'.length)      // 14
console.log(''.length)                             // 0
```

23

charAt()方法

24

字符串的常用方法

- “方法”就是能够打点调用的函数，字符串有丰富的方法

方法	功能
charAt()	得到指定位置字符
substring()	提取子串
substr()	提取子串
slice()	提取子串
toUpperCase()	将字符串变为大写
toLowerCase()	将字符串变为小写
indexOf()	检索字符串

25

charAt()方法

- charAt()方法可以得到指定位置的字符，位置序号从0开始

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.charAt(0)      // '我'
'我喜欢JS，我也喜欢HTML'.charAt(11)     // 'T'
'我喜欢JS，我也喜欢HTML'.charAt(74)     // ''
```

26

可以直接书写[]得到某位字符

- 可以直接书写[]得到某位字符，和数组非常类似

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'[0]      // '我'
'我喜欢JS，我也喜欢HTML'[11]     // 'T'
'我喜欢JS，我也喜欢HTML'[74]     // ''
```

27

substring()、substr()和slice()方法

28

substring()、substr()和slice()

- substring()、substr()和slice()方法都用来截取字符串的子字符串，但是有细微差别
- 这三个字符串的区别，面试常考

29

substring()

- substring(a, b)方法得到从a开始到b结束（不包括b处）的子串

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.substring(3, 5)        // 'JS'
'我喜欢JS，我也喜欢HTML'.substring(10, 14)      // 'HTML'
```

30

substring()

- substring(a, b)方法如果省略第二个参数，返回的子串会一直到字符串的结尾

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.substring(6)   // '我也喜欢HTML'
```

31

substring()

- substring(a, b)中，a可以大于b，数字顺序将自动调整为小数在前

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.substring(5, 3)       // 'JS'
```

32

substr()

- substr(a, b)中，将得到从a开始的长度为b的子串

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.substr(3, 2)      // 'JS'
```

33

substr()

- substr(a, b)中，b可以省略，表示到字符串结尾

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.substr(3)      // 'JS，我也喜欢HTML'
```

34

substr()

- substr(a, b)中，a可以是负数，表示倒数位置

'我喜欢JS，我也喜欢HTML'
0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.substr(-4, 2)      // 'HT'
```

35

slice()

36

slice()

- slice(a, b)方法得到从a开始到b结束（不包括b处）的子串，是的，和substring()是相同的

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.slice(3, 5)      // 'JS'
```

37

slice()

- slice(a, b)的参数可以是负数

'我喜欢JS，我也喜欢HTML'

-5 -4 -3 -2 -1

```
'我喜欢JS，我也喜欢HTML'.slice(-4, -1)      // 'HTM'
```

38

slice()

- slice(a, b)中，参数a必须小于参数b

'我喜欢JS，我也喜欢HTML'

0 1 2 3 4 5 6 7 8 9 10 11 12 13

```
'我喜欢JS，我也喜欢HTML'.slice(5, 2)      // ''
```

39

substring()、substr()和slice()总结

- substring和slice功能基本一致，都是得到从a开始到b结束（不包括b）的子串，区别：
  - ① substring可以自动交换两个参数位置，而slice()不行；
  - ② slice的参数可以是负数，而substring()不行
- substr(a, b)中参数b是子串长度，而不是位置编号

40

toUpperCase()和toLowerCase()方法

41

toUpperCase()和toLowerCase()方法

- toUpperCase() 转为大写
- toLowerCase() 转为小写

```
'I love you'.toUpperCase();          // 'I LOVE YOU'
'ABcdEfg'.toLowerCase();             // 'abcdefg'
```

42

indexOf()方法

43

indexOf()方法

44

indexOf()方法

- indexOf()方法返回某个指定的字符串值在字符串中首次出现的位置
- 如果要检索的字符串值没有出现，则该返回-1

```
'abcdeb'.indexOf('b')          // 1
'abcdeb'.indexOf('de')         // 3
'abcdeb'.indexOf('m')          // -1
```

45

indexOf()方法

- indexOf()方法还可以接收第二个参数，表示从第几位开始寻找

```
'abcdeb'.indexOf('b', 2)   // 5
```

表示从2号位置开始寻找

46

boolean（布尔）类型

47

布尔

boolean

英 /ˈbuːliən/ 美 /ˈbuːliən/ 全球(美国)

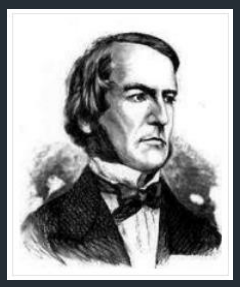
简明 牛津 新牛津 韦氏 NEW 例句 百科

adj. 布尔数学体系的

48

布尔

- 布尔（Boole·George）英国19世纪数学家及逻辑学家。



49

布尔

- 布尔型值只有两个：true和false，分别表示真和假
- 不要小瞧布尔型值，它们在编程中出现频率非常高

```
console.log(typeof true);    // boolean
console.log(typeof false);   // boolean
```

50

布尔

- 逻辑计算的结果是布尔型值

```
console.log(3 > 8); // false
console.log(3 < 8); // true
```

51

undefined类型

52

还记得undefined是嘛么?

- 一个没有被赋值的变量的默认值是undefined
- 有意思的是，undefined的类型也是undefined
- 即：undefined又是值，又是一种类型，这种类型只有它自己一个值

```
console.log(typeof undefined);  // undefined
```

53

null类型

54

null类型是什么

55

null类型是什么

- null表示“空”，确切的说是“空对象”
- 当我们需要将对象销毁、数组销毁或者删除事件监听时，通常将它们设置为null

```
box.onclick = null;
```

这条语句看不懂正常，今后学习DOM时会介绍

56

用typeof检测null结果是object

- 使用typeof检测null值，结果是object，这点尤其要注意

```
console.log(typeof null);    // object
```

57

typeof检测结果

	类型	typeof检测结果	值举例
基本类型值	数字	number	5
	字符串	string	'爱前端'
	布尔	boolean	true
	undefined	undefined	undefined
	null	object	null
引用类型值	数组	object	[33, 44, 11]
	对象	object	{a: 1, b: 2}
	函数	function	function(){}
	正则表达式	object	/\d{3}/
	……		

58